

1 INTRODUCTION

1.1. INTRODUCTION

1.2. AIMS AND OBJECTIVE OF THE WORK

1.3. BRIEF LITERATURE REVIEW

1.4. PROBLEM DEFINITION

1.5. PLAN OF THE WORK

1.1. Introduction

StrideX is a cross-platform mobile application designed to revolutionize the way people interact with outdoor physical activity by introducing a gamified, location-based territory claiming mechanic paired with comprehensive fitness tracking and community competition. In an era where sedentary lifestyles and screen-time dependency are growing public health concerns, traditional fitness tracking applications often fail to provide long-term engagement beyond basic step counts and distance logs. StrideX addresses this challenge by transforming the physical act of walking or running into a strategic, competitive, and visually rewarding game experience, enriched with daily fitness monitoring, hydration tracking, wellness features, and a unique Indian twist called "Mohalla Wars" — locality-based team competition.

At its core, the application empowers users to claim real-world geographic areas as their own territories simply by moving through them. As a user begins a run, the app activates real-time GPS tracking, plotting their movement path as a visible polyline on an interactive Google Map. The system continuously monitors the user's trajectory for self-intersections — when the user's path crosses over itself, forming a closed loop, the application automatically detects this event, extracts the enclosed polygon, calculates its area, and registers it as a claimed territory in the database. The app now computes actual distance covered using Turf.js and estimates calories burned for each run.

Claimed territories are rendered as colored polygons directly on the map, with each user's territories displayed in a unique color derived from their identity. Users authenticate via Clerk (supporting Google OAuth and email/password) and select their mohalla (locality/ward) during registration using GPS-based reverse geocoding. Individual territory claims aggregate into the user's mohalla score, fueling the "Mohalla Wars" leaderboard — a locality-based team competition that serves as the application's Indian USP. A rank progression system inspired by Indian culture (Pathfinder, Strider, Trailblazer, Zamindar, Raja, Maharaja) rewards users as they claim more territory.

The backend is powered by Convex, a modern real-time Backend-as-a-Service (BaaS) platform that provides instant data synchronization across all connected clients. This means that when one user claims a territory, all other users see the update in real-time on their maps. The application now features five database tables (users, mohallas, runs, territory, dailyLogs) and a comprehensive set of serverless functions for user management, mohalla CRUD, fitness logging, and leaderboard computation.

The application features a five-tab navigation structure: Map (territory claiming), Dashboard (daily stats, weekly charts, fitness goals), Leaderboard (global rankings and Mohalla Wars), Wellness (hydration tracker, mental math mini-game, daily tips, reminders), and Profile (rank badge, personal stats, run history). Additionally, StrideX includes a Developer Mode that allows testing via a draggable map marker without requiring physical GPS movement.

By merging fitness tracking with competitive gamification, community-based mohalla competition, daily wellness monitoring, and real-time geospatial computing, StrideX aims to inspire users to step outside, explore their neighborhoods, and engage in a novel form of location-based recreation that is physically beneficial, intellectually stimulating, and uniquely Indian in its community-oriented competitive design.

1.2. Aims and Objective of the Work

1.2.1. Aims of “StrideX”

StrideX is a mobile application designed to gamify outdoor physical activity by combining real-time GPS tracking with a territory claiming mechanic. Based on the project’s design and implementation, its primary aims are:

- **Gamified Fitness Engagement:** To provide a game-like incentive for users to walk, run, and explore outdoor environments by rewarding physical movement with virtual territory ownership and rank progression.
- **Real-Time Territory Visualization:** To allow users to see their claimed territories and those of other players rendered as colored polygons on a live, interactive map, creating a shared competitive landscape.
- **Automated Loop Detection:** To replace manual input with an intelligent, automated system that detects when a user’s movement path forms a closed loop and automatically claims the enclosed area as territory.
- **Cross-Platform Accessibility:** To deliver a seamless and consistent experience across both Android and iOS devices using a single, unified codebase built with React Native and Expo.
- **Community-Based Competition (Mohalla Wars):** To enable locality-based team competition where individual territory claims aggregate into a shared mohalla score, fostering neighborhood pride and collective fitness motivation — the application's Indian USP.
- **Comprehensive Fitness & Wellness Tracking:** To provide daily fitness parameters (distance, calories, duration), hydration tracking, mental math exercises, wellness tips, and workout reminders — transforming the app into a holistic fitness companion beyond territory claiming.

In short, StrideX aims to make physical activity more engaging, competitive, and community-driven by turning every outdoor run or walk into a strategic territory-claiming mission that represents the user's mohalla in locality-wide competition.

1.2.2. Objectives of “StrideX”

StrideX enables users to claim geographic territories through physical movement, combining fitness tracking with real-time competitive gameplay and community-based mohalla competition. The specific objectives are:

- **Implement Real-Time GPS Tracking:** Develop a robust location tracking system using Expo Location APIs that continuously captures the user's GPS coordinates during a run session with high accuracy and minimal battery drain.
- **Automate Closed-Loop Detection:** Implement a geospatial algorithm using Turf.js that analyzes the user's path in real-time, detects self-intersections that form closed polygons, filters them by a minimum area threshold (50 sq meters), and registers them as claimed territories.
- **Persist Run and Territory Data:** Utilize Convex as a real-time backend to store run histories (path coordinates, timestamps, distance) and territory records (polygon coordinates, center points, area, ownership) with instant synchronization across all users.
- **Visualize Territories on Map:** Render all claimed territories as colored polygon overlays on a Google Maps interface, with ownership-based color coding (green for self, red for others, blue for newly claimed in the current session).
- **Provide Developer Testing Mode:** Include a dev mode toggle that replaces live GPS tracking with a draggable map marker, enabling efficient testing of all app features without requiring physical outdoor movement.
- **Implement User Authentication: Integrate Clerk SDK with Google OAuth and email/password sign-in, connected to Convex via authenticated queries and mutations.**
- **Build Mohalla Wars System: Implement locality detection via reverse geocoding, mohalla assignment during registration, and aggregate leaderboards where neighborhoods compete based on collective territory claimed.**
- **Develop Multi-Screen Navigation: Build a five-tab navigation structure (Map, Dashboard, Leaderboard, Wellness, Profile) using React Navigation with bottom tabs.**
- **Track Daily Fitness Parameters: Auto-populate distance, duration, and estimated calories from completed runs; provide a hydration tracker, daily goals, and weekly progress charts on the Dashboard screen.**
- **Implement Wellness Features: Provide a mental math mini-game, rotating daily wellness tips, and workout/hydration reminder support via expo-notifications.**
- **Implement Rank Progression: Create a six-tier rank system (Pathfinder, Strider, Trailblazer, Zamindar, Raja, Maharaja) based on total area claimed, with rank badges displayed on profiles and leaderboards.**

These objectives together aim to make StrideX an efficient, engaging, and technically robust platform for gamified location-based fitness tracking with a unique Indian community competition element.

1.3. Brief Literature Review

The development of StrideX is grounded in existing research and innovations across the fields of location-based gaming, fitness gamification, and geospatial computing in mobile environments.

1.3.1. Location-Based Gaming & Gamification

The concept of using real-world geographic locations as a game mechanic was popularized by applications such as Niantic's Ingress (2012) and Pokémon GO (2016). Research by Althoff et al. (2016) published in the Journal of Medical Internet Research demonstrated that Pokémon GO led to significant increases in physical activity among its users, with an average increase of 1,473 daily steps over a 30-day period. This established a clear precedent that location-based games can be effective tools for promoting physical health. However, these applications rely on pre-placed virtual objects at fixed locations, limiting the user's agency in defining the game world. StrideX advances this paradigm by allowing users to create the game world themselves through their movement patterns.

1.3.2. Fitness Tracking & Mobile Health

Traditional fitness applications like Strava, Nike Run Club, and Google Fit have been instrumental in digitizing exercise routines. A study by Middelweerd et al. (2014) in the International Journal of Behavioral Nutrition and Physical Activity found that while fitness apps increase short-term awareness, long-term engagement often declines due to "tracking fatigue"—users lose motivation when the only feedback is numerical statistics. The integration of game elements (points, competition, visual rewards) has been shown to counteract this effect. StrideX incorporates these gamification principles by providing immediate visual feedback (territory polygons on the map) and competitive elements (seeing other players' territories).

1.3.3. Geospatial Computing in Mobile Applications

The computational challenge of detecting self-intersecting paths and extracting polygons in real-time on a mobile device is addressed through the Turf.js library. Research in computational geometry, particularly the Bentley-Ottmann algorithm for detecting line segment intersections, provides the theoretical foundation. Turf.js implements these algorithms in an optimized JavaScript package suitable for mobile runtimes. Studies such as "Efficient Algorithms for Geographic Information Systems" (de Berg et al., 2008) demonstrate that real-time geospatial processing is feasible on consumer-grade mobile hardware when using optimized libraries and appropriate data structures.

1.4. Problem Definition

The current landscape of fitness and location-based applications faces several critical challenges that StrideX seeks to address:

- **Low Long-Term Engagement in Fitness Apps:** Traditional fitness tracking apps primarily offer numerical statistics such as distance, pace, and calories burned. Research consistently shows that this passive data presentation leads to user disengagement over time. Users lack a compelling reason to continue running beyond personal discipline, as the app provides no evolving, dynamic incentive.
- **Limited Player Agency in Location-Based Games:** Existing location-based games like Pokémon GO and Ingress rely on pre-determined Points of Interest (POIs) placed by the developers. Users cannot create or modify the game world through their own actions. This creates geographic inequality where users in rural or less-populated areas have significantly fewer in-game elements to interact with.
- **Absence of Movement-Driven World Building:** No mainstream application currently allows users to “paint” the real world with their movement and claim physical geographic space as a game mechanic. The concept of transforming a user’s running path into a polygon-based territory that persists on a shared map is a novel interaction paradigm.
- **Technical Barriers in Real-Time Geospatial Processing:** Detecting closed loops in a GPS path in real-time on a mobile device presents computational challenges. GPS noise, signal drift, and the high frequency of coordinate updates require efficient algorithms for path smoothing, self-intersection detection, and polygon extraction—all while maintaining a responsive user interface.
- **Absence of Community-Based Fitness Competition:** No existing fitness or territory app ties individual physical activity to neighborhood or locality-level pride. Users exercise in isolation without any mechanism to represent their community. The concept of mohalla-based team competition — where a user's runs contribute to their neighborhood's collective score — is entirely absent from the market.

1.5. Plan of the Work

To ensure the successful development and deployment of StrideX, the project work is divided into the following strategic phases:

Phase 1: Requirement Analysis & Feasibility

Objective: Identify the core problems in existing fitness and location-based apps and define the project scope.

Activities:

- Reviewing literature on location-based gaming, fitness gamification, and geospatial computing.
- Defining the core game mechanic: users claim territory by forming closed loops while running.

- Determining the technical stack: React Native (Expo), Convex, Turf.js, Google Maps.
- Evaluating feasibility of real-time closed-loop detection on mobile hardware.

Phase 2: System Design

Objective: Architect the database schema, component structure, and user interface.

Activities:

- Database Design: Defining the Convex schema with five tables — users, mohallas, runs, territory, and dailyLogs — with appropriate indexes for efficient querying.
- UI/UX Design: Designing a five-screen tab-based interface (Map, Dashboard, Leaderboard, Wellness, Profile) with authentication screens and mohalla onboarding flow.
- Algorithm Design: Designing the closed-loop detection algorithm using Turf.js with a minimum area threshold of 50 square meters, plus distance calculation using `turf.length()` and calorie estimation algorithms.

Phase 3: Implementation (Development)

Objective: Coding the functional modules of the application.

Activities:

- Location Tracking Module: Developing the `useLocationTracker` custom hook for real-time GPS.
- Geometry Engine: Implementing `findClosedLoops()` and `smoothPath()` in `geometry.ts`.
- Map Rendering: Building the HomeScreen with `MapView`, polygon overlays with per-user coloring, polyline rendering, and live distance tracking.
- Backend Integration: Connecting to Convex with `saveRun` mutations (with auto-updating user stats, mohalla aggregates, and daily logs), leaderboard queries, fitness logging, and user/mohalla management functions.
- Developer Mode: Implementing a toggle-able dev mode with a draggable marker.
- Authentication Module: Integrating Clerk SDK with Google OAuth and email/password, including sign-in, sign-up, and email verification screens.
- Mohalla Wars System: Implementing reverse geocoding for locality detection, mohalla selection during onboarding, and aggregate leaderboard computation.
- Navigation & Screens: Building five-tab navigation with React Navigation, Dashboard (daily stats, weekly charts), Leaderboard (global + Mohalla Wars), Wellness (hydration tracker, mental math, tips), and Profile (rank, stats, history).
- Rank System: Implementing six-tier rank progression (Pathfinder through Maharaja) with automatic rank updates on each run and visual rank badges on profiles.
- Demo Data Seeding: Creating a seed script with 8 mohallas across Gandhinagar/Ahmedabad and 12 demo users with territories for presentation.

Phase 4: Testing & Validation

Objective: Ensure system reliability, accuracy, and performance.

Activities:

- Unit Testing: Verifying the closed-loop detection algorithm with various path configurations.
- Integration Testing: Testing the full flow from GPS tracking to territory saving to map rendering.
- Field Testing: Conducting outdoor tests with real GPS data under real-world conditions.
- Dev Mode Testing: Validating the draggable marker simulation produces consistent results.

Phase 5: Deployment & Maintenance

Objective: Prepare the app for distribution and plan future improvements.

Activities:

- Building production bundles for Android and iOS using Expo's build service.
- Deploying the Convex backend to production.
- Gathering user feedback for future features: offline support, territory conflict resolution with geometric operations, push notification scheduling, social challenges.

2 TECHNOLOGY AND LITERATURE REVIEW

2.1. TECHNOLOGIES

2.2. LITERATURE REVIEW

2.1. Technologies

2.1.1. Frontend Technologies (Client-Side)

- **React Native (v0.81.5):** The primary framework for building the mobile user interface. React Native enables the development of truly native mobile applications for both Android and iOS using a single JavaScript/TypeScript codebase. It uses native UI components rather than web views, ensuring high performance and a native look-and-feel.
- **React (v19.1.0):** The underlying UI library that React Native is built upon. Provides the component-based architecture, hooks system (useState, useEffect), and declarative rendering paradigm used throughout the application.
- **Expo (~v54.0.32):** A managed development platform built on top of React Native that simplifies the build, deployment, and testing workflow. Expo provides expo-location (~v19.0.8) for accessing the device's GPS hardware, Expo CLI for project management, and Over-the-Air (OTA) update capabilities.
- **TypeScript (v5.9.2):** A statically-typed superset of JavaScript used across the entire codebase. TypeScript provides compile-time type checking, enhanced IDE support, and improved code maintainability—critical for complex geospatial data structures like coordinate arrays and polygon objects.
- **react-native-maps (v1.20.1):** A mapping component library that wraps native Google Maps (Android) and Apple Maps (iOS) SDKs. Used for rendering the interactive MapView, drawing path trails (Polyline), displaying territory polygons (Polygon), and placing markers including the draggable marker in dev mode.
- **@clerk/clerk-expo (v2.19.30):** Authentication SDK for React Native/Expo that provides Google OAuth and email/password sign-in, session management via secure token storage, and integration with Convex's authenticated queries through the ConvexProviderWithClerk bridge.
- **@react-navigation/native + @react-navigation/bottom-tabs (v7.x):** The navigation framework providing a five-tab bottom tab navigator (Map, Dashboard, Leaderboard, Wellness, Profile) and a native stack navigator for authentication screens (Sign In, Sign Up).
- **expo-secure-store (~v15.0.8):** Provides encrypted key-value storage on the device, used to securely persist Clerk authentication tokens between app sessions.
- **expo-web-browser (~v15.0.10):** Handles OAuth redirect flows by opening the system browser for Google sign-in and returning control to the app upon completion.
- **expo-notifications (v0.32.16):** Enables local push notifications for workout reminders and periodic hydration reminders scheduled during active hours.

2.1.2. Backend Technologies (Server-Side)

- **Convex (v1.31.6):** A real-time Backend-as-a-Service (BaaS) platform that serves as the complete backend infrastructure. Convex provides Real-Time Queries (the useQuery hook automatically re-renders the UI whenever underlying data changes), Mutations (server-side functions for writing data with automatic validation), Schema Enforcement (strongly-typed database schemas), Serverless Functions (backend logic runs without managing servers), Auto-Generated TypeScript Types for end-to-end type safety, and native integration with Clerk for authenticated queries via ConvexProviderWithClerk.

2.1.3. Database System

- **Convex Document Database:** A serverless, real-time document database that stores data as JSON-like documents within tables. Each document has a system-generated `_id` field and a `_creationTime` timestamp. The database enforces a schema defined in `convex/schema.ts`. The database now consists of five tables — `users`, `mohallas`, `runs`, `territory`, and `dailyLogs` — with defined indexes for efficient querying on `userId`, `ownerId`, `clerkId`, `mohallaId`, `totalArea`, `city`, and composite `user+date`.

2.1.4. Geospatial & Location Services

- **Turf.js (v7.3.3):** A comprehensive geospatial analysis library for JavaScript. Used for `lineString()` to convert GPS coordinates to GeoJSON, `lineIntersect()` to detect self-intersection points, `unkinkPolygon()` to split self-intersecting polygons into valid simple polygons, `area()` to calculate geodesic area in square meters, and `centroid()` to compute the geographic center of each polygon.
- **expo-location (~v19.0.8):** Provides the interface to the device's GPS hardware: permission handling (`requestForegroundPermissionsAsync`), continuous location updates (`watchPositionAsync`) with configurable accuracy, and coordinate data in WGS 84 format.
- **Google Maps Platform:** Provides the base map tiles rendered by `react-native-maps` with `PROVIDER_GOOGLE`. Requires a Google Maps API key. Additionally, the Expo Location reverse geocoding API (`Location.reverseGeocodeAsync`) is used during registration to detect the user's locality/ward for mohalla assignment — a key component of the Mohalla Wars feature.

2.2. Literature Review

2.2.1. Contextual Background

- **The Sedentary Lifestyle Crisis:** The World Health Organization (WHO) identifies physical inactivity as the fourth leading risk factor for global mortality, accounting

for approximately 3.2 million deaths annually. Despite the proliferation of fitness tracking applications, studies show that only 10-20% of users remain active after 6 months of initial download (Krebs & Duncan, 2015).

- **The Gamification Solution:** Gamification—the application of game-design elements in non-game contexts—has emerged as a proven strategy for increasing user engagement. Deterding et al. (2011) established the foundational framework demonstrating that elements such as points, visual progress indicators, and competitive mechanics significantly improve user motivation in health applications.
- **Location-Based Services (LBS) Evolution:** The maturation of smartphone GPS hardware and widespread availability of mapping APIs have made it technically feasible to build sophisticated location-aware applications. The global LBS market was valued at \$23.14 billion in 2023 with strong projected growth.

2.2.2. Existing Solutions & Market Analysis

- **Pokémon GO (Niantic, 2016):** The most commercially successful location-based game with over 1 billion downloads. Requires players to walk to specific locations to interact with virtual creatures. However, it relies on pre-placed Points of Interest, creating geographic inequality.
- **Strava (2009):** A leading fitness social network with over 100 million users. Introduced the “segment” concept for route-based competition. However, it does not offer any territory-claiming or world-building mechanic.
- **Ingress (Niantic, 2012):** A territory-control game where teams capture “portals” at real-world locations. Territory is tied to fixed portal locations rather than dynamically created by user movement.

None of the above solutions allow users to dynamically create territory through their own movement patterns—the core innovation of StrideX.

2.3. Problem Statement & Gaps

- **Engagement Gap:** Despite the availability of hundreds of fitness tracking applications, long-term user retention remains critically low. The primary gap is the absence of a dynamic, evolving incentive system that rewards physical movement with tangible, persistent, and visible outcomes.
- **Agency Gap:** Existing location-based games limit players to interacting with developer-placed content. Users cannot shape the game world through their own actions.
- **Geographic Equity Gap:** Games like Pokémon GO concentrate game elements in urban centers, disadvantaging rural and suburban users. A system where users generate content through movement would function equally well in any geographic setting.

- **Technical Gap:** Real-time closed-loop detection in GPS paths on mobile devices is an underexplored technical problem. Integration of Turf.js into a mobile-first, real-time application with GPS noise handling represents a novel engineering challenge.
- **Community Gap:** No existing fitness or territory application ties individual exercise to neighborhood-level collective competition. Users lack a mechanism to represent their locality or community through their physical activity, missing an opportunity for social motivation rooted in local pride.

2.4. The Role of Gamified Intervention (The Project's Contribution)

- **From Tracking to Playing:** StrideX shifts fitness tracking from a passive data-recording activity to an active, game-like experience where every step has strategic significance.
- **User-Generated Game World:** Unlike pre-placed content models, StrideX allows users to dynamically create the game world through their movement, creating an ever-evolving competitive landscape.
- **Visual & Competitive Motivation:** The immediate visual feedback of seeing a territory polygon appear on the map provides a dopamine-driven reward cycle that numerical statistics cannot match.
- **Technical Innovation:** Real-time detection of self-intersecting GPS paths, extraction of enclosed polygons, filtering by minimum area, and persistence to a shared real-time database represents a novel technical pipeline.
- **Geographic Equality:** Since territory is generated by user movement rather than pre-placed POIs, StrideX functions equally well in urban, suburban, and rural environments.
- **Community-Driven Competition (Mohalla Wars):** StrideX introduces locality-based team competition where individual territory claims aggregate into a mohalla's collective score. This "Mohalla Wars" system — where neighborhoods compete on a shared leaderboard — represents a novel Indian-centric USP that combines fitness gamification with community pride, absent from all existing applications.
- **Holistic Fitness Platform:** Beyond territory claiming, StrideX provides daily fitness tracking (distance, calories, duration), hydration monitoring, mental math exercises, wellness tips, and a six-tier rank progression system — transforming a single-mechanic game into a comprehensive fitness companion.

3 SYSTEM REQUIREMENTS STUDY

3.1. USER CHARACTERISTICS

3.2. HARDWARE AND SOFTWARE REQUIREMENTS

3.3. ASSUMPTIONS AND DEPENDENCIES

3.1. User Characteristics

StrideX is designed for three distinct user groups, each with specific roles and technical aptitudes.

3.1.1. Runners & Fitness Enthusiasts (Primary Users)

- **Role:** The primary content creators who generate territory by physically moving through the real world.

Characteristics:

- Demographics: Health-conscious individuals aged 16-45, varying in fitness levels.
- Technical Expertise: Low to Medium. The app uses an intuitive five-tab navigation interface with clear visual cues.
- Frequency of Use: Regular (daily or several times per week).

Key Privileges:

- Start and stop run tracking sessions.
- View real-time movement path on the map.
- Automatically claim territories when closed loops are detected.
- View all territories (own in green, others' in unique per-user colors).
- View personal dashboard with daily stats, weekly progress, and fitness goals.
- Track hydration and access wellness features (mental math, daily tips).

3.1.2. Casual Gamers & Explorers (Secondary Users)

- **Role:** Users drawn to the competitive and exploratory aspects of the game rather than fitness goals.

Characteristics:

- Demographics: Gamers and explorers of all ages interested in location-based gaming.
- Technical Expertise: Medium. Familiar with mobile gaming and map-based apps.
- Frequency of Use: Sporadic to regular, driven by competitive motivation.

3.1.3. Mohalla Representatives (Tertiary Users)

- **Role:** Users primarily motivated by representing their locality (mohalla) on the Mohalla Wars leaderboard, driven by neighborhood pride.
- **Characteristics:**
- Demographics: Community-oriented individuals who identify with their neighborhood. Frequency of Use: Regular, driven by mohalla leaderboard standing.

3.2. Hardware and Software Requirements

3.2.1. Hardware Requirements

Since StrideX is a mobile application with a cloud-based backend, the hardware requirements are specific to the end-user device.

Client-Side (User Device)

- Device: Android smartphone (v5.0+) or iPhone (iOS 13+).
- Processor: Quad-core 1.5 GHz or faster.
- RAM: Minimum 2 GB.
- GPS Hardware: Built-in GPS receiver with fine location accuracy.
- Internet Connection: 3G/4G/5G or Wi-Fi (mandatory for real-time sync).
- Storage: Minimum 100 MB free space.

Server-Side (Cloud Infrastructure)

- Fully managed by Convex's cloud infrastructure.
- No dedicated server hardware required from the development team.
- Convex handles auto-scaling, database hosting, and serverless function execution.

3.2.2. Software Requirements

Development Environment

- Operating System: Windows 10/11.
- Code Editor: Visual Studio Code / Cursor IDE.
- Runtime: Node.js (v18+ LTS) with npm.
- Mobile Testing: Expo Go app or Android Emulator / iOS Simulator.
- Backend CLI: Convex CLI (npv convex dev).

Technology Stack (Runtime Environment)

- Mobile Framework: React Native (v0.81.5) with Expo (~v54.0.32).
- Language: TypeScript (v5.9.2).
- Backend: Convex (v1.31.6).
- Maps: Google Maps SDK via react-native-maps (v1.20.1).
- Geospatial Engine: Turf.js (v7.3.3).
- Location API: expo-location (~v19.0.8).
- Authentication: Clerk SDK (@clerk/clerk-expo v2.19.30) with Google OAuth.
- Navigation: React Navigation (@react-navigation/native v7.x, bottom-tabs, native-stack).
- Notifications: expo-notifications (v0.32.16) for local push reminders.

3.3. Assumptions and Dependencies

3.3.1. Assumptions

- **GPS Availability:** It is assumed that the user's device has a functioning GPS receiver and adequate satellite coverage (typically outdoors).
- **Internet Connectivity:** It is assumed that the user has an active internet connection during the run session. Offline mode is not currently supported.
- **Outdoor Usage:** It is assumed that users will primarily use the app outdoors where GPS accuracy is sufficient (3-10 meters).
- **Authenticated Users:** **The application requires user authentication via Clerk (Google OAuth or email/password). Each user has a unique profile with mohalla affiliation, rank, and cumulative statistics.**
- **Honest Movement:** The system assumes users are physically moving to generate paths. Dev mode exists only for testing.

3.3.2. Dependencies

- **Convex Cloud Service:** The application depends on Convex's cloud infrastructure for all backend operations. Downtime would render the app unable to save runs or display territories.
- **Google Maps API Key:** Map rendering depends on a valid Google Maps API key. Exceeding the free-tier quota would disable map rendering.
- **Expo Location Permissions:** The app depends on the user granting foreground location permissions.
- **Turf.js Library:** The closed-loop detection logic depends entirely on Turf.js. Breaking changes would require code updates.
- **React Native Maps Compatibility:** The react-native-maps library must remain compatible with installed React Native and Expo versions.
- **Clerk Authentication Service:** **User authentication depends on Clerk's cloud service for OAuth flows, session management, and JWT token issuance. Clerk downtime would prevent new sign-ins (existing sessions remain cached locally via expo-secure-store).**

4 SYSTEM DIAGRAMS

4.1. ENTITY-RELATIONSHIP DIAGRAM

4.2. CLASS DIAGRAM

4.3. USE CASE DIAGRAM

4.4. SEQUENCE DIAGRAM

4.5. ACTIVITY DIAGRAM

4.6. DATA FLOW DIAGRAM

4.1. Entity-Relationship Diagram

An Entity-Relationship Diagram (ERD) is a visual representation used in database design to illustrate the structure and relationships between various entities or objects within a system. It employs entities (representing real-world objects or concepts), attributes (properties of these entities), and relationships (connections between entities) to depict how data is organized and interrelated in a database.

The StrideX ERD consists of five primary entities: User (identified by clerkId, with name, email, rank, and cached stats), Mohalla (identified by name, city, state, with aggregate stats and member count), Run (a session containing GPS path, distance, duration, calories, and timestamps, related to User via userId), Territory (a claimed polygon with coordinates, center, area, and ownership, related to User via ownerId), and DailyLog (daily fitness metrics including distance, calories, water intake, related to User via userId). User has a many-to-one relationship with Mohalla, one-to-many with Run, Territory, and DailyLog.

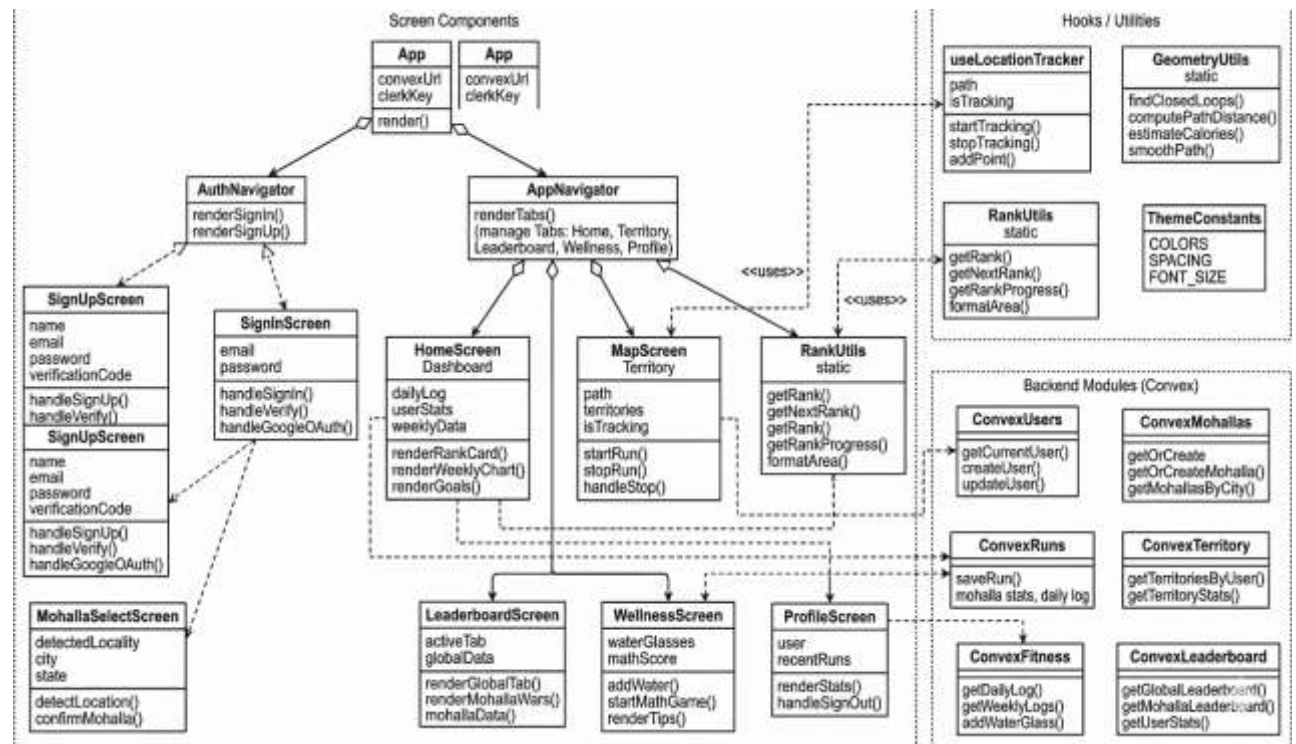


Fig4.1: Entity-Relationship Diagram

4.2. Class Diagram

A class diagram is a visual representation in UML that depicts the structure and relationships of classes within a software system. The StrideX class diagram includes: App (root component with ClerkProvider and ConvexProviderWithClerk), AppNavigator (bottom tab navigator with 5 screens), AuthNavigator (stack navigator for SignIn/SignUp), HomeScreen (map UI with territory coloring and live distance), DashboardScreen (daily stats, weekly chart, goals), LeaderboardScreen (global + Mohalla Wars tabs), WellnessScreen (hydration tracker, mental math, tips), ProfileScreen (rank badge, stats, run history), MohallaSelectScreen (onboarding with reverse geocoding), useLocationTracker (custom hook for GPS state management), GeometryUtils (static utility with findClosedLoops, computePathDistance, estimateCalories), RankUtils (getRank, getRankProgress, formatArea), and ConvexBackend modules (users.ts, mohallas.ts, runs.ts, territory.ts, fitness.ts, leaderboard.ts, seed.ts).

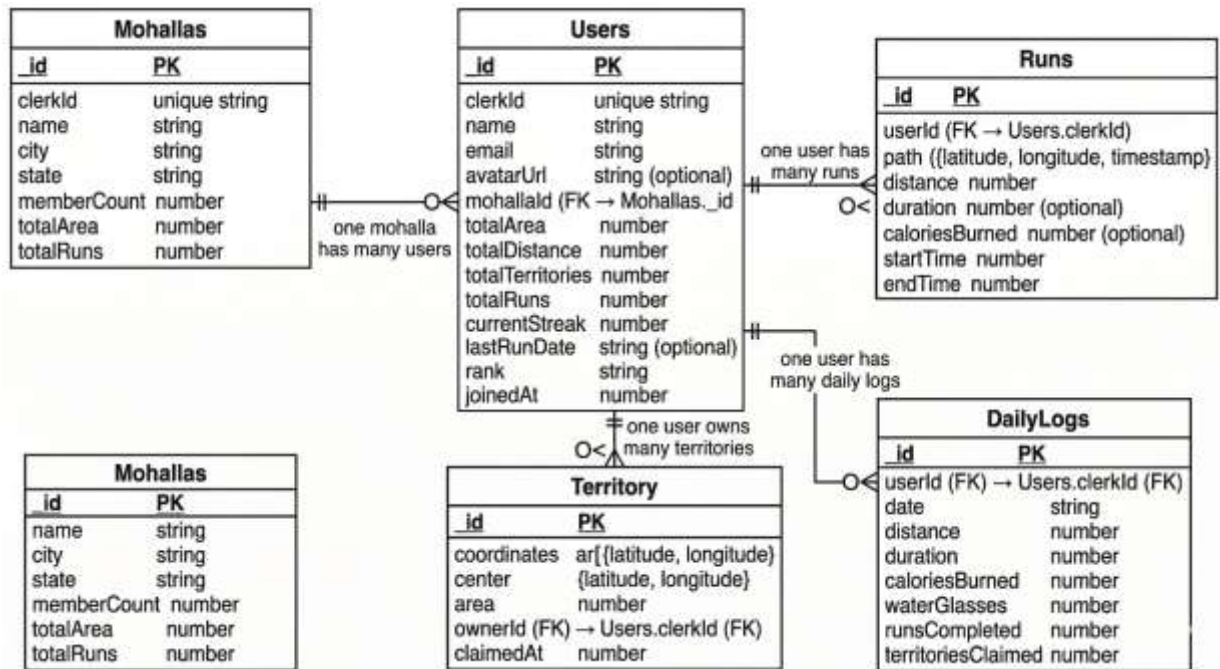


Fig4.2: Class Diagram

4.3. Use Case Diagram

A use case diagram depicts the interactions between actors and the system. Actors in StrideX include Runner (primary actor), GPS Hardware (system actor), Convex Backend (system actor), and Clerk Auth (system actor). Use cases include: Sign In / Sign Up (via Google OAuth or email), Select Mohalla (with reverse geocoding), Start Run Session, Track Location in Real-Time, View Movement Path, Detect Closed Loop (automated), Claim Territory (automated), Stop Run Session, Save Run to Database, View Dashboard (daily stats, weekly charts, goals), View Global Leaderboard, View Mohalla Wars Leaderboard, Track Hydration (add/remove water glasses), Play Mental Math Game, View Wellness Tips, View Profile & Rank, View Run History, Toggle Dev Mode, Simulate Movement, and Sign Out.

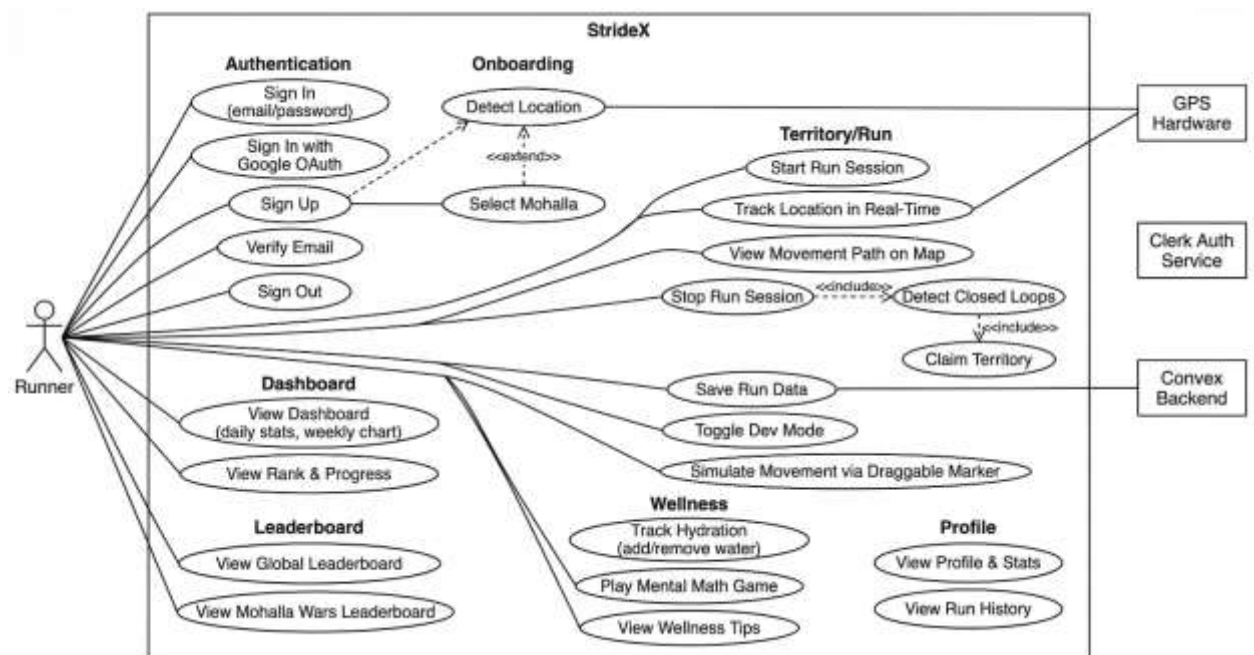


Fig4.3: Use Case Diagram

4.4. Sequence Diagram

A sequence diagram shows the sequence of messages passed between objects over time. The primary flows include: (1) Authentication Flow: User opens app → ClerkProvider checks session → If not signed in: AuthNavigator shows SignIn/SignUp → User authenticates via Clerk → ConvexProviderWithClerk establishes authenticated connection → App checks for user profile → If new: MohallaSelectScreen detects location via reverse geocoding → User confirms mohalla → createUser mutation saves profile. (2) Territory Claiming Flow: Runner presses Start Run → HomeScreen calls useLocationTracker.startTracking() → expo-location provides GPS updates → path accumulates with live distance calculation → Runner presses Stop Run → computePathDistance() and estimateCalories() calculate metrics → findClosedLoops() processes the path via Turf.js → saveRun mutation saves run, territories, updates user stats, mohalla aggregates, and daily log → new territories render on the map with per-user coloring.

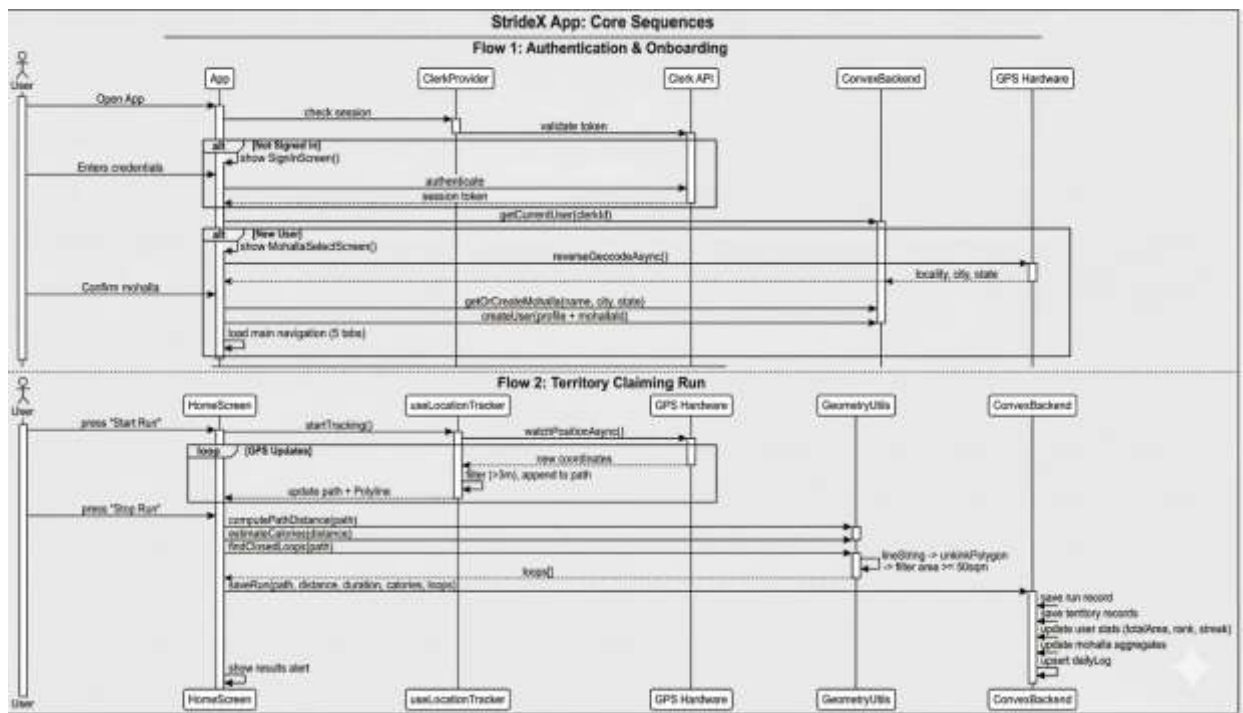


Fig4.4: Sequence Diagram

4.5. Activity Diagram

Activity diagram models the flow from one activity to another. The StrideX activity flow: User opens app → Clerk checks authentication state → If not signed in: show auth screens (Sign In / Sign Up) → After auth: check if user profile exists in Convex → If new user: show MohallaSelectScreen (detect location, select mohalla, create profile) → Load main app with bottom tab navigation → User can navigate between Map, Dashboard, Leaderboard, Wellness, Profile tabs → On Map tab: User presses Start Run → App requests GPS permission → Subscribe to GPS updates → For each update, append to path and render Polyline with live distance → User presses Stop Run → Compute distance and calories → Run findClosedLoops() → If loops found, filter by area ≥ 50 sqm → Save run, territories, update user stats and mohalla aggregates to Convex → Render new territories on map → Display results alert.

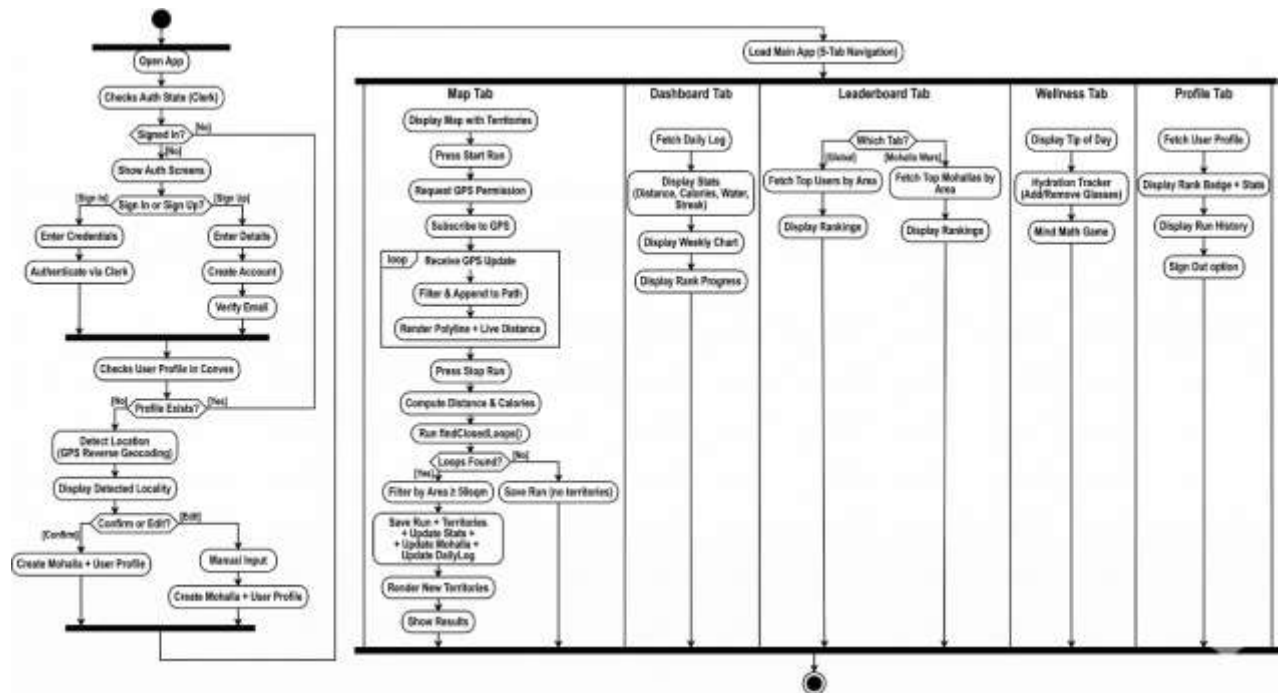


Fig4.5: Activity Diagram

4.6. Data Flow Diagram

A Data Flow Diagram (DFD) depicts the flow of data within a system. It uses symbols to illustrate how data moves from input sources through processes to output destinations.

4.6.1. Context-Level-0 DFD

The Context-Level-0 DFD shows the entire StrideX system as a single process. External entities: Runner (provides GPS coordinates, commands, authentication credentials, and fitness inputs; receives map with territories, dashboard stats, leaderboard rankings, and wellness content), GPS Hardware (provides location data), Clerk Auth Service (handles authentication and provides user identity), and Convex Cloud (exchanges user profiles, mohalla data, run records, territory records, daily logs, and leaderboard computations).

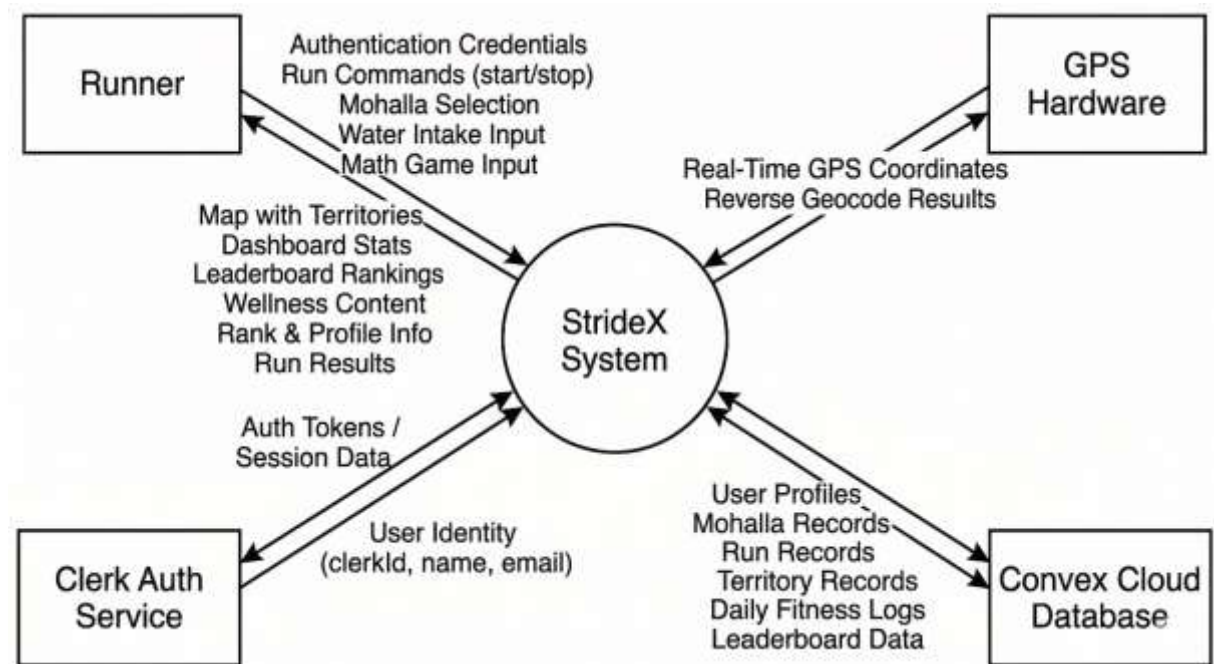


Fig4.6.1: Context-Level-0 Data Flow Diagram

4.6.2. Context-Level-1 DFD

The Level-1 DFD breaks the system into major processes: Authentication & User Management, Mohalla Assignment, Location Tracking, Path Management, Loop Detection, Territory Management, Fitness Logging, Leaderboard Computation, Wellness Services, and Map Rendering, with the Convex Database (5 tables) as the central data store.

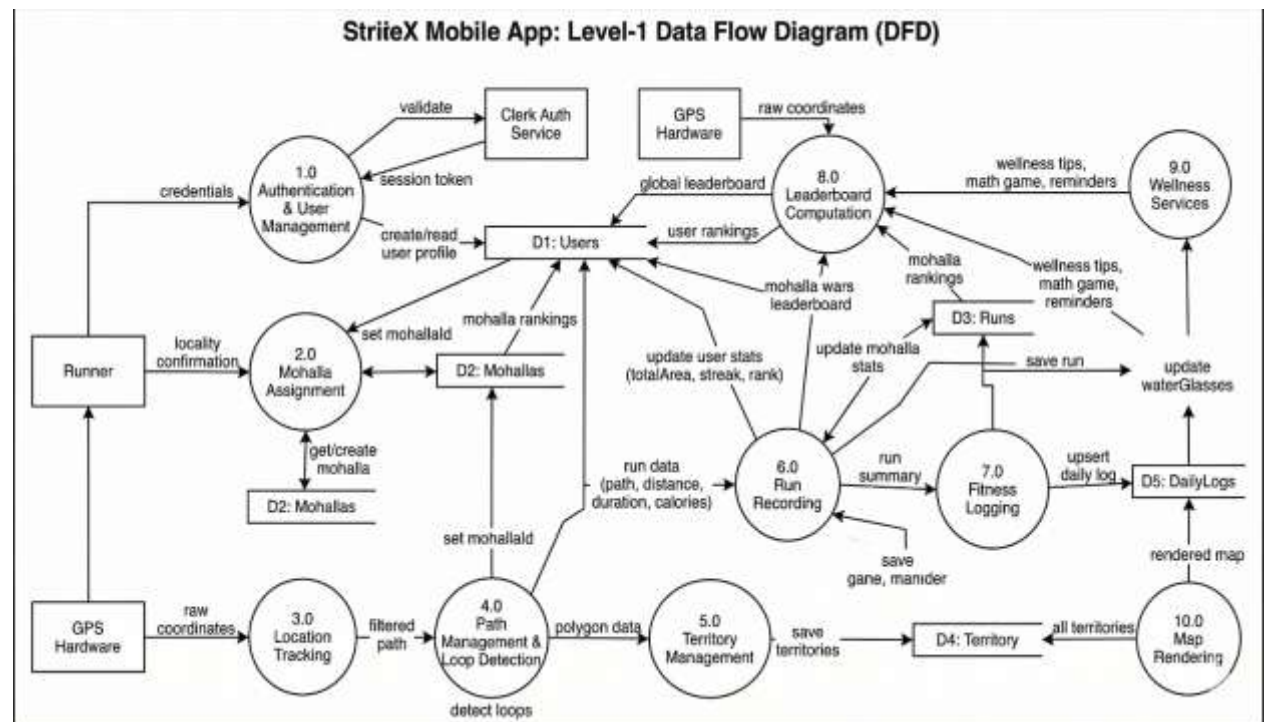


Fig4.6.2: Context-Level-1 Data Flow Diagram

5 DATA DICTIONARY

5.1. OVERVIEW

5.2. DATABASE SCHEMA

5.3. DATA INTEGRITY AND INDEXING

5.1. Overview

StrideX utilizes Convex, a serverless real-time document database, to store and manage all application data. Unlike traditional relational databases (MySQL, PostgreSQL), Convex stores data as JSON-like documents within defined tables. The database schema is enforced through a TypeScript schema definition file (convex/schema.ts), and all interactions with the database are performed through type-safe serverless functions (queries and mutations) defined in convex/users.ts, convex/mohallas.ts, convex/runs.ts, convex/territory.ts, convex/fitness.ts, convex/leaderboard.ts, and convex/seed.ts. Convex handles CRUD operations with real-time synchronization, meaning any data change is instantly reflected across all connected clients.

5.2. Database Schema

The database consists of 5 tables designed to handle user management, mohalla (locality) tracking, run session recording, territory claim management, and daily fitness logging.

5.2.1. Table: Runs

This table stores the details of each run session completed by a user, including the full GPS path, computed distance, duration, estimated calories burned, and timestamps. The distance field is now computed from actual path coordinates using Turf.js length(), and caloriesBurned is estimated at approximately 60 kcal per kilometer.

Column Name	Data Type	Constraints	Description
_id	ID (auto)	PRIMARY KEY, Auto-generated	Unique identifier for each run record.
_creationTime	Number (auto)	Auto-generated	Unix timestamp of record creation.
userId	String	NOT NULL	Identifier of the user who performed the run.
path	Array of Objects	NOT NULL	Array of {latitude, longitude} coordinate objects.
distance	Number (Float)	NOT NULL	Total distance covered in meters.
startTime	Number	NOT NULL	Unix timestamp of run start.
endTime	Number	NOT NULL	Unix timestamp of run end.

Table 1: Runs Table Schema

5.2.2. Table: Territory

This is the core game-state table that stores every territory claimed by all users. Each row represents a single polygon on the map.

Column Name	Data Type	Constraints	Description
_id	ID (auto)	PRIMARY KEY, Auto-generated	Unique identifier for each territory.
_creationTime	Number (auto)	Auto-generated	Unix timestamp of record creation.
coordinates	Array of Arrays	NOT NULL	Polygon vertices as [longitude, latitude] pairs.
center	Object	NOT NULL	Geographic center {latitude, longitude}.
area	Number (Float)	NOT NULL	Geodesic area in square meters (min: 50).
ownerId	String	NOT NULL	Identifier of the claiming user.
claimedAt	Number	NOT NULL	Unix timestamp of territory claim.

Table 2: Territory Table Schema

5.2.3. Table: Users

Column Name	Data Type	Constraints	Description
_id	ID (auto)	PRIMARY KEY, Auto-generated	Unique identifier for each user.
clerkId	String	UNIQUE, NOT NULL	Identifier from the Clerk authentication service.
name	String	NOT NULL	Full name of the user.
email	String	UNIQUE, NOT NULL	Email address associated with the account.
mohallaId	ID	FOREIGN KEY	Reference to the Mohalla the user belongs to.
totalArea	Number (Float)	DEFAULT 0	Total territory area claimed by the user in square meters.
totalDistance	Number (Float)	DEFAULT 0	Total running distance recorded by the user (meters).

Table 3: Users Table Schema

5.2.4. Table: Mohallas

Column Name	Data Type	Constraints	Description
_id	ID (auto)	PRIMARY KEY, Auto-generated	Unique identifier for each mohalla (locality/ward).
name	String	NOT NULL	Name of the mohalla/locality.
city	String	NOT NULL	City in which the mohalla is located.
state	String	NOT NULL	State where the mohalla exists.
memberCount	Number	DEFAULT 0	Number of registered users in this mohalla.
totalArea	Number (Float)	DEFAULT 0	Aggregate territory area claimed by members (square meters).
totalRuns	Number	DEFAULT 0	Total number of runs completed by all members of the mohalla.

Table 4: Mohallas Table Schema

5.2.5. Table: DailyLogs

Column Name	Data Type	Constraints	Description
_id	ID (auto)	PRIMARY KEY, Auto-generated	Unique identifier for each daily log entry.
userId	ID	FOREIGN KEY, NOT NULL	Identifier of the user associated with the log.
date	String	NOT NULL	Date of the activity record (YYYY-MM-DD).
distance	Number (Float)	DEFAULT 0	Total distance covered by the user on that day (meters).
duration	Number	DEFAULT 0	Total running duration for the day (seconds).

caloriesBurned	Number	DEFAULT 0	Estimated calories burned during runs.
runsCompleted	Number	DEFAULT 0	Number of runs completed on that day.

Table 5: DailyLogs Table Schema

5.3. Data Integrity and Indexing

- **Primary Keys:** Every document in Convex automatically receives a unique `_id` field that serves as the primary key for efficient lookups.
- **Schema Validation:** The Convex schema enforces strict type checking at the database level. Non-conforming documents are rejected.
- **Indexing:** The runs table has an index on `userId` (`by_user`). The territory table has an index on `ownerId` (`by_owner`). The users table has indexes on `clerkId` (`by_clerkId`), `mohallaId` (`by_mohalla`), and `totalArea` (`by_totalArea`). The mohallas table has indexes on `city` (`by_city`) and `totalArea` (`by_totalArea`). The dailyLogs table has a composite index on `userId + date` (`by_user_date`).
- **Data Types:** Arrays store variable-length GPS paths and polygon coordinates. Float64 is used for all coordinate and area values for geographic precision.
- **Relationships:** The users table is linked to mohallas via `mohallaId` (many-to-one). The runs and territory tables are linked to users via `userId/ownerId`. The dailyLogs table is linked to users via `userId` with a composite key on `userId + date`. Territories created during a run are saved atomically within the same `saveRun` mutation, which also updates the user's cached stats, mohalla aggregates, and daily log in a single transactional operation.

6 RESULT, DISCUSSION AND CONCLUSION

6.1. RESULT

6.2. CONCLUSION

6.1. Result

The StrideX application has been successfully developed, tested, and expanded into a comprehensive fitness-gamification platform. The following functional outcomes have been achieved:

- **1. Real-Time GPS Tracking:** The `useLocationTracker` hook successfully captures real-time GPS coordinates with high accuracy using expo-location, rendering a smooth polyline on the map that traces the user's movement path. Live distance is now computed and displayed during the run using `Turf.js length()`.
- **2. Closed-Loop Detection:** The `findClosedLoops()` algorithm, powered by `Turf.js`, accurately detects self-intersecting paths and extracts enclosed polygons. Testing confirmed that the system correctly identifies simple loops, complex figure-8 patterns, and ignores paths that do not form closed regions.
- **3. Territory Claiming and Persistence:** Detected territories are successfully saved to the Convex database via the `saveRun` mutation. The mutation atomically updates the user's cached statistics (`totalArea`, `totalDistance`, `totalTerritories`, `totalRuns`, `streak`), the user's mohalla aggregate stats, and the daily fitness log — all in a single transaction. Territories appear on the map in real-time with per-user coloring.
- **4. Developer Mode:** The dev mode toggle successfully replaces live GPS tracking with a draggable marker, allowing full testing without requiring physical outdoor movement.
- **5. User Authentication:** Clerk SDK integration with Google OAuth and email/password sign-in is fully implemented. The app detects authentication state and routes users through sign-in, sign-up (with email verification), or the main app accordingly. A dev mode fallback allows testing without Clerk configuration.
- **6. Mohalla Wars System:** During registration, the app detects the user's locality via reverse geocoding and presents it for confirmation. Users can also manually enter their mohalla. Territory claims automatically aggregate into mohalla scores, displayed on the dedicated Mohalla Wars leaderboard tab.
- **7. Multi-Screen Navigation:** A five-tab bottom navigation (Map, Dashboard, Leaderboard, Wellness, Profile) provides seamless access to all features. Authentication uses a separate stack navigator with Sign In and Sign Up screens.
- **8. Leaderboards:** The Global leaderboard ranks users by total area with rank badges and mohalla affiliations. The Mohalla Wars leaderboard ranks neighborhoods by collective area with member counts, fostering locality-based competition.
- **9. Dashboard & Fitness Tracking:** The Dashboard displays today's distance, calories burned, water intake, and streak count. A custom weekly bar chart

shows 7-day distance trends. Progress bars track daily goals for distance (3 km) and hydration (8 glasses).

- **10. Wellness Features:** The hydration tracker allows adding/removing water glasses with visual progress indicators. The mental math mini-game presents 60-second timed arithmetic challenges with score tracking. Rotating daily wellness tips cover hydration, nutrition, stretching, sleep, and mental health.
- **11. Rank Progression:** The six-tier rank system (Pathfinder, Strider, Trailblazer, Zamindar, Raja, Maharaja) is fully implemented. Ranks are automatically updated based on total area claimed and displayed on profiles, leaderboards, and the Dashboard with progress bars toward the next rank.
- **12. Demo Data:** A seed script successfully populates 8 mohallas across Gandhinagar and Ahmedabad with 12 demo users, territories, and daily logs, enabling full demonstration of all features during presentation.

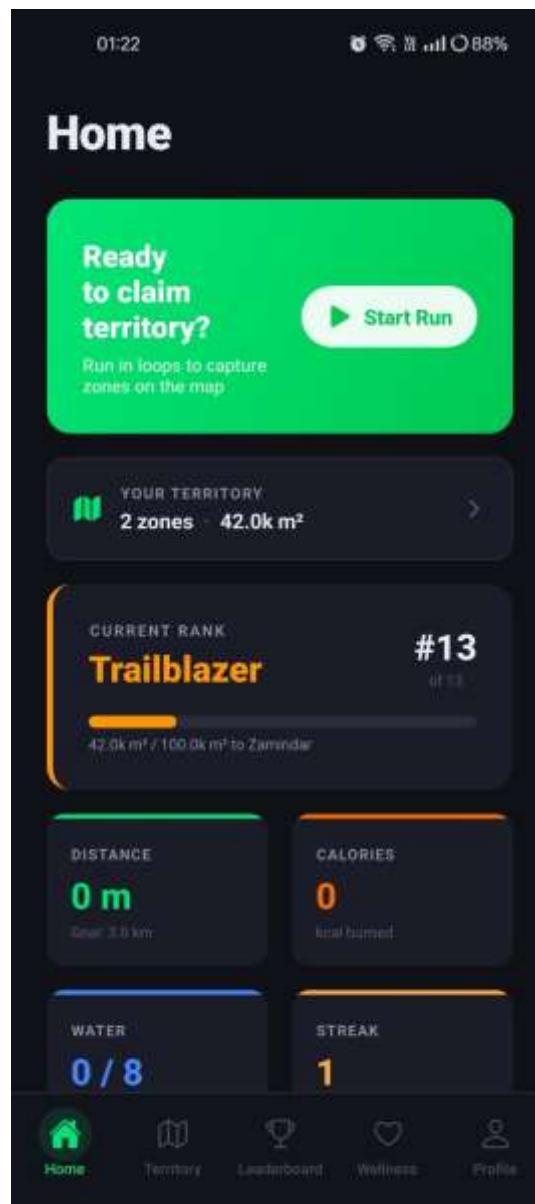


Fig6.1: App Screenshot – Home Screen (Dashboard)



Fig6.2: App Screenshot – Territory Map with Multi-User Territories



Fig6.3: App Screenshot – Leaderboard (Global Rankings)

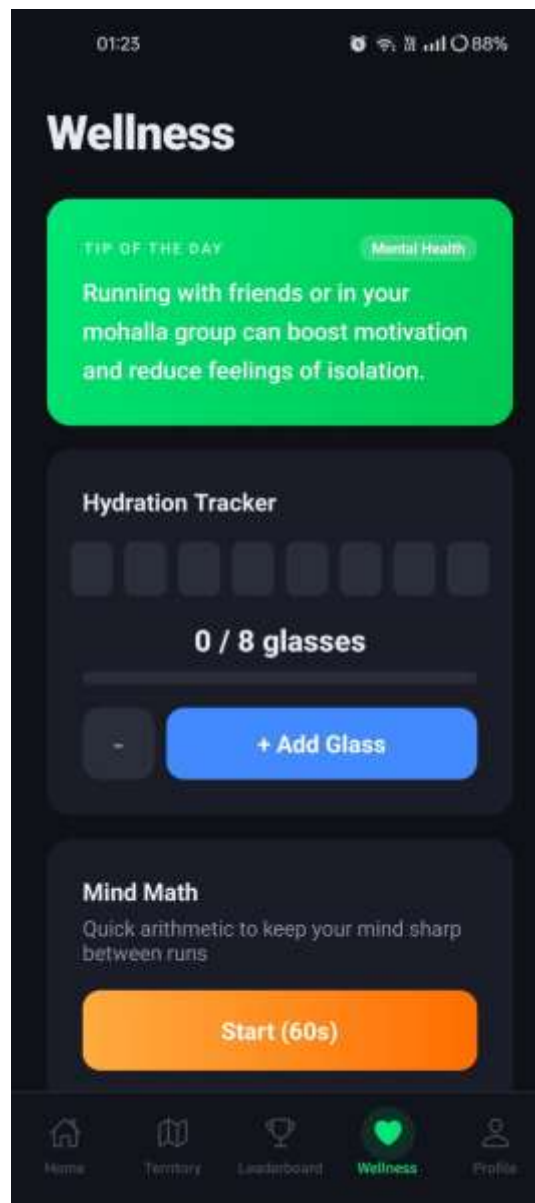


Fig6.4: App Screenshot – Wellness Screen (Hydration & Mind Math)

6.2. Conclusion

StrideX successfully demonstrates that gamified territory claiming, combined with community-based mohalla competition and comprehensive fitness tracking, can serve as a compelling incentive for outdoor physical activity. By integrating real-time GPS tracking, computational geometry algorithms, user authentication, locality-based team competition, daily fitness monitoring, and wellness features into a cohesive multi-screen application, the project creates a novel user experience where physical movement directly translates into visible, competitive, and persistent game outcomes tied to neighborhood pride.

The project validates several key technical achievements:

- Real-time self-intersection detection in GPS paths is feasible on mobile devices using Turf.js, with actual distance computation and calorie estimation.
- Convex's real-time synchronization provides an effective backbone for shared-world multiplayer experiences without the complexity of traditional WebSocket management.
- The React Native + Expo stack, combined with Clerk authentication and React Navigation, delivers a consistent, multi-screen cross-platform experience from a single TypeScript codebase.
- The Mohalla Wars system — locality-based team competition with reverse geocoding and aggregate leaderboards — represents a novel Indian-centric contribution to the fitness gamification space, absent from all existing applications.

Future enhancements planned for subsequent iterations include:

- Territory conflict resolution using geometric operations (`turf.difference`) to handle overlapping claims between users.
- Offline support with sync-on-reconnect capability for uninterrupted run tracking.
- Scheduled push notifications for workout and hydration reminders via expo-notifications.
- Social features: friend system, direct challenges, and team-based territory wars beyond mohalla competition.
- Expanded fitness tracking: step counting via device pedometer, workout plans, and integration with health APIs (Google Fit / Apple Health).
- Festival events: time-limited territory challenges tied to Indian festivals (Diwali Run, Independence Day Challenge) for seasonal engagement.

By bridging the gap between passive fitness tracking and active location-based gaming, and by introducing the novel concept of mohalla-based community competition, StrideX contributes to the growing body of work demonstrating that gamification rooted in local culture and community pride is a powerful tool for promoting sustained physical activity and outdoor exploration.

7 BIBLIOGRAPHY

7.1. REFERENCES

7.1. References

This section lists the academic papers, technical documentation, and online resources referenced during the development and research of StrideX.

7.1.1. Technical Documentation

React Native Documentation : <https://reactnative.dev/docs/getting-started>

Expo Documentation : Available: <https://docs.expo.dev/>

Convex Documentation : Available: <https://docs.convex.dev/>

Turf.js Documentation : Available: <https://turfjs.org/docs/>

Google Maps Platform Documentation : developers.google.com/maps/documentation

react-native-maps Documentation : <https://github.com/react-native-maps/react-native-maps>

TypeScript Documentation : <https://www.typescriptlang.org/docs/>

Clerk Documentation : <https://clerk.com/docs>

React Navigation Documentation : <https://reactnavigation.org/docs/getting-started>

Expo Notifications Documentation : <https://docs.expo.dev/versions/latest/sdk/notifications/>

Expo Secure Store Documentation : <https://docs.expo.dev/versions/latest/sdk/securestore/>

7.1.2. Similar Systems (Case Studies)

Pokémon GO – Niantic : <https://pokemongolive.com/>.

Strava – Fitness Social Network : <https://www.strava.com/>.

Nike Run Club : <https://www.nike.com/nrc-app>.

Turf Territory (Turf Game) : <https://turfgame.com/>.

Intrvl (Inspiration App) : <https://www.intrvl.com/>